

```
"""
```

falsification_validator.py – GRUS Audit Pipeline Stage 8

Specification: GRUS-STD-001-v1.0 Section 3 Stage 8; Clause 5

Verifies pending predictions have valid falsification specifications.

Published by: Green Recursive Utility Service LLC

```
"""
```

```
from typing import Dict, List
```

```
from dataclasses import dataclass, field
```

```
@dataclass
```

```
class FalsificationValidationResult:
```

```
    passed: bool
```

```
    valid_specs: List[Dict] = field(default_factory=list)
```

```
    invalid_specs: List[Dict] = field(default_factory=list)
```

```
    failure_reason: str = ""
```

```
REQUIRED_FALSIFICATION_KEYWORDS = ["if", "exceeds", "below", "above", "outside",  
    "fails", "deviates", "diverges", "less than", "greater than", "<", ">", "#"]
```

```
def validate_falsification_spec(spec: str) -> bool:
```

```
    if not spec or len(spec) < 20: return False
```

```
    spec_lower = spec.lower()
```

```
    return any(kw in spec_lower for kw in REQUIRED_FALSIFICATION_KEYWORDS)
```

```
def validate_all_falsifications(manifest: Dict) -> FalsificationValidationResult:
```

```
    valid, invalid = [], []
```

```
    for p in manifest.get("predictions", []):
```

```
        if p.get("status") != "pending": continue
```

```
        spec = p.get("falsification_condition", "")
```

```
        if validate_falsification_spec(spec):
```

```
            valid.append({"id": p.get("id"), "spec": spec})
```

```
        else:
```

```
            invalid.append({"id": p.get("id"),
```

```
                           "reason": "falsification_condition missing or insufficient (must
```

```
specify a measurable failure threshold)}")
```

```
    if invalid:
```

```
        return FalsificationValidationResult(passed=False, valid_specs=valid,
```

```
invalid_specs=invalid,
```

```
        failure_reason=f"{len(invalid)} pending predictions have invalid falsification specs  
(Clause 5)")
```

```
    return FalsificationValidationResult(passed=True, valid_specs=valid, invalid_specs=[])
```

```
if __name__ == "__main__":
```

```
    import sys, yaml, os
```

```
    if len(sys.argv) < 2:
```

```
        print("Usage: python falsification_validator.py <repo_path>"); sys.exit(0)
```

```
    with open(os.path.join(sys.argv[1], "GRUS_MANIFEST.yaml")) as f:
```

```
        m = yaml.safe_load(f)
```

```
r = validate_all_falsifications(m)
if r.passed:
    print(f"[PASS] {len(r.valid_specs)} pending predictions have valid falsification specs")
else:
    print(f"[FAIL] {r.failure_reason}")
    for inv in r.invalid_specs: print(f"  {inv['id']}: {inv['reason']}")
    sys.exit(1)
```